instances out of them when the form is submitted. That code needs to go somewhere, and the unit tests you're about to write against that code need to know where that place is. This is where the design thinking comes in the TDD process.

No matter where you put the actual coding logic, Rails will still insist on the existence of a controller, so you have the separate decision of how (or whether) to test whatever logic winds up in the controller itself.

Let's start with the business logic; I'll come back to the controller.

Three locations are commonly used for business logic that responds to user input beyond the common "pass the `params` hash to `ActiveRecord#create`" Rails behavior. Here are the options:

- Put the extra logic in the controller. This is often the Rails core team's preferred method, and if there isn't much logic it works perfectly fine. In my experience this location doesn't work as well for complex logic. It's challenging to test, awkward to refactor, and difficult to share if that becomes an issue. It also becomes confusing if there is more than one complicated action in the controller.
- Put the extra logic in the associated model, often at least partially in a class method. This was my go-to move for years. It's somewhat easier to test, but still kind of awkward to refactor—Ruby class method semantics are a pain. It also makes the model more complicated and doesn't really help manage actions that legitimately span multiple models, as this one does since it's creating a `Project` and multiple `Tasks`.
- Create a separate class to encapsulate the logic and workflow. This tends to be my first choice these days. It's the easiest to test and the best able to manage complexity changes as they come. The main downside is you wind up with a lot of little classes, but I don't mind having a lot of little pieces anyway.

Prescription 8 Placing business logic outside Rails classes makes that logic easier to test and manage.

So, you're creating a new class. I'll stress here that this design is not the only way to go, and if you feel the complexity of this particular action doesn't warrant its own class, that's fine. There is no consistent generic name for a logic class like this. Let's call it a workflow class. Other names you might see in use for different kinds of logic classes include action, service, context, use case, concern, and factory.